



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251189
Nama Lengkap	Alicia Luna Santoso
Minggu ke / Materi	13/ Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Link Github : https://github.com/alc357/71251189_alicia.git

TUPLE IMMUTABLE

Bentuk tuple menyerupai list. Nilai yang disimpan dalam tuple dapat berupa apapun dan diberikan indeks bilangan bulat (integer). Perbedaan utamanya adalah sifat tuple yang immutable (tidak bisa dirubah). Tuple dapat dibandingkan (compare, menggunakan method `__eq__()` atau `__cm__()`) dan bersifat hashable (tidak pernah berubah selama hidupnya dengan method `__hash__()`) sehingga dapat dimasukkan dalam list sebagai key pada dictionary python. Hashability membuat semua objek yang dapat digunakan sebagai kamus kunci dan mengatur anggota, karena struktur data menggunakan nilai hash secara internal.

Sintaks penulisan tuple :

```
>>> t = 'a','b','c','d','e'
```

Sintaks lain penulisan tuple dengan menggunakan tanda kurung :

```
>>> t = ('a','b','c','d','e')
```

Tuple dengan satu element, ditambahkan (,), di belakang penulisannya karena jika tidak akan dianggap sebagai string :

```
>>> t1 = ('a',)    >>> t1 = ('a')
>>> type(t1)      >>> type(t1)
<class 'tuple'>  <class 'str'>
```

Dapat juga kita menggunakan fungsi built-in tuple. Ketika tidak diisi dengan argument apapun, secara langsung akan membentuk tuple kosong.

```
>>> t = tuple()
>>> print(t)
()
```

Jika argument berupa urutan (string, list, atau tuple), akan mengembalikan nilai tuple dengan elemen-elemen yang berurutan.

```
>>> t = tuple('alpro')
>>> print(t)
('a', 'l', 'p', 'r', 'o')
```

Tuple merupakan nama dari constructor, kita tidak dapat menggunakannya sebagai variabel. Sebagian besar operator yang bekerja pada list bekerja pada tuple. Tanda kurung kotak [] menandakan indeks element dari tuple.

```
>>> t = ('a','b','c','d','e')    >>> t = ('a','b','c','d','e')
>>> print(t[3])                 >>> print(t[0:3])
d                                ('a', 'b', 'c')
```

Tuple bersifat immutable (elemen tidak dapat berubah). Jika anda berusaha mengubah tuple, maka akan error.

```
>>> t[0] = 'A'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    import platform
    ^^^^^
TypeError: 'tuple' object does not support item assignment
```

Elemen pada tuple tidak dapat dirubah tetapi dapat diganti dengan elemen lain.

```
>>> t = ('A',) + t[1:]
>>> print(t)
('A', 'b', 'c', 'd', 'e')
...
```

MEMBANDINGKAN TUPLE

Operator perbandingan (compare) dapat bekerja pada tuple dan model sekuensial lainnya (list, dictionary, set). Cara kerjanya dengan membandingkan elemen pertama dari setiap sekuensial yang ada. Jika ditemukan kesamaan, akan berlanjut ke elemen berikutnya. Proses berlangsung terus hingga ditemukan perbedaan.

```
>>> (0,1,2) < (0,3,4)
True
>>> (0,4,6) < (0,3,4)
False
```

Fungsi sort pada python bekerja dengan cara yang sama. Tahap pertama akan melakukan pengurutan berdasarkan elemen pertama. Pada kasus tertentu akan mengurutkan berdasarkan elemen kedua dan sebagainya. Fitur ini disebut dengan DSU (decorate, Sort, Undercorate)

1. Decorate – urutan (sekuensial) membangun daftar tuple dengan satu atau lebih key pengurutan sebelum elemen dari urutan (sekuensial).
2. Sort – list tuple menggunakan sort (fungsi bawaan dari python)
3. Undercote – melakukan ekstraksi pada elemen yang telah diurutkan pada satu sekuensial.

Contohnya, kita mempunyai sebuah kalimat dan akan melakukan pengurutan kata dalam kalimat tersebut dari yang paling panjang ke yang paling pendek.

```
1 kalimat = 'but soft what light in yonder window breaks'
2 dafkata = kalimat.split()
3 t = list()
4 for kata in dafkata :
5     t.append((len(kata), kata))
6
7 t.sort (reverse = True)
8
9 urutan = list ()
10 for length, kata in t :
11     urutan.append(kata)
12
13 print(urutan)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\prak_alpro\random> & "C:/Users/alc_357/AppData/Local/Programs/Python/f
['yonder', 'window', 'breaks', 'light', 'what', 'soft', 'but', 'in']
PS C:\prak_alpro\random> 
```

Gambar 1.1 : contoh penggunaan tuple untuk mengurutkan kata

Looping pertama akan membuat daftar tuple yang berisi kata sesuai dengan panjangnya. Fungsi sort akan membandingkan elemen pertama dari list panjang kata yang ada dan kemudian menuju ke elemen kedua jika kondisi sesuai. Keyword reverse = True digunakan untuk melakukan urutan secara terbalik (descending). Looping kedua membuat daftar tuple dalam sesuai urutan alfabet diurutkan berdasarkan panjangnya. Pada contoh, empat kata dalam kalimat akan diurutkan secara alfabet terbalik. Kata “what” akan muncul sebelum “soft” dalam list. Kata-kata yang muncul diurutkan sebagai list dengan urutan panjang kata dari yang paling panjang ke kata yang paling pendek.

PENUGASAN TUPLE

Salah satu fitur dari python adalah kemampuan untuk memiliki tuple di sisi kiri dari statement penugasan. Hal ini mengizinkan untuk menetapkan lebih dari satu variabel pada sisi sebelah kiri secara berurutan. Contohnya ketika ada dua daftar elemen yang merupakan urutan. Kita akan mencoba menetapkan elemen pertama dan kedua dari urutan tersebut ke dalam variabel x dan dalam satu statement. Berikut contohnya :

```
>>> m = ['have', 'fun']
>>> x,y = m
>>> x
'have'
>>> y
'fun'
```

```
>>> m = ['have', 'fun']
>>> x = m[0]
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
```

```
>>> m = ['have', 'fun']
>>> (x,y) = m
>>> x
'have'
>>> y
'fun'
```

Tuple memungkinkan untuk menukar nilai variabel dalam satu statement

```
>>>> a, b = b, a
```

Dari contoh, dapat dilihat bahwa kedua statement adalah tuple. Bagian kiri merupakan tuple dari variable dan bagian kanan merupakan tuple dari expressions. Tiap nilai bagian kanan diberikan ke masing-masing variable di sebelah kiri. Semua expressions pada bagian kiri dievaluasi sebelum diberikan penugasan. Yang perlu diperhatikan adalah jumlah antara sisi kiri dan sisi kanan harus sama.

```
>>> a, b = 1, 2, 3
ValueError: too many values to unpack
```

Pada sisi sebelah kanan biasanya terdapat data sekuensial string, list, tuple. Contohnya, kita akan membagi Alamat email menjadi user name dan domain seperti berikut

```
>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')
```

Nilai yang dikembalikan dari split terdiri dari dua elemen yang dipisahkan tanda “@”, elemen pertama berisi username dan elemen berikutnya adalah domain.

```
>>> print(username)
didanendya
>>> print(domain)
ti.ukdw.ac.id
```

DICTIONARIES AND TUPLE

Dictionaries mempunyai metode items untuk mengembalikan nilai list dari tuple dimana tiap tuple nya merupakan key value pair (pasangan kunci dan nilai).

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]
```

Item yang dikembalikan nilainya tidak berurutan. List dari tuple adalah list, dan tuple membandingkannya sehingga kita dapat melakukan pengurutan (sort) pada tuple. Melakukan

konversi dictionary dan list tuple merupakan cara untuk menampilkan isi dictionary yang diurutkan berdasarkan kuncinya.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

List yang muncul diurutkan secara ascending berdasarkan alfabet dan key value.

MULTIPENUGASAN DENGAN DICTIONARIES

Kombinasi antara items, tuple assignment dan for akan menghasilkan kode tertentu pada keys dan values dari dictionary dalam satu loop

```
for key, val in list(d.items()):
    print(val, key)
```

Terdapat dua iterasi variabel karena item mengembalikan list dari tuple dan key. Val merupakan penugasan tuple yang melakukan iterasi berulang melalui pasangan key-value pada dictionary. Pada setiap iterasi yang melalui loop, baik key dan value akan bergerak maju menuju pasangan key-value berikutnya pada dictionary (masih dalam urutan hash).

Jika dilakukan penggabungan dari kedua Teknik, kita dapat melakukan pencetakan isi dictionary yang diurutkan berdasarkan nilai yang disimpan di setiap pasangan key-value. Untuk melakukannya, langkah pertama dengan membuat list tuple di mana masing-masing tuple beranggotakan (value, key). Method item akan memberikan list tuple (value, key), dan akan melakukan pengurutan berdasarkan value, bukan key. Setelah daftar list dengan value-key tuple terbentuk, lalu kita mengurutkan list dengan urutan terbalik dan melakukan pencetakan list baru yang disortir.

KATA YANG SERING MUNCUL

```
2 import string
3 fhand = open('romeo-full.txt')
4 counts = dict()
5 for line in fhand:
6     line = line.translate(str.maketrans('', '', string.punctuation))
7     line = line.lower()
8     words = line.split()
9     for word in words:
10         if word not in counts:
11             counts[word] = 1
12         else:
13             counts[word] += 1
14
15 # urutkan dictionary by value
16 lst = list()
17 for key, val in list(counts.items()):
18     lst.append((val, key))
19
20 lst.sort(reverse=True)
21
22 for key, val in lst[:10]:
23     print(key, val)
```

TUPLE SEBAGAI KUNCI DICTIONARIES

Ketika kita ingin membuat composite key yang digunakan pada dictionary, kita dapat menggunakan tuple sebagai key. Misalnya ketika membuat direktori telepon yang memetakan pasangan last-name, first-name ke nomor telepon .

```
for last, first in directory:
    print(first, last, directory[last,first])

>>> last = 'nendya'
>>> first = 'dida'
>>> number = '088112266'
>>> directory = dict()
>>> directory[last, first] = number
>>> for last, first in directory:
...     print(first, last, directory[last,first])
...
dida nendya 088112266
>>>
```

Sumber : Modul 13-Tipe Data Tuple

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/alc357/71251189_alicia.git

SOAL 1

Source Code :

```
inp_user = eval(input("Masukan tuple angka : "))

if not isinstance(inp_user, tuple) :

    print("inputmu bukan tuple heii")

else :

    sama = True

    if len(inp_user) == 0 :

        print(sama)

        print("(Tapi Tuple mu kosong)")

    elif len(inp_user) == 1 :

        print(sama)

        print("(Tapi Tuple mu hanya memiliki 1 angka)")

    else :

        for i in range (len(inp_user)-1) :

            if inp_user[i] != inp_user[i+1] :

                sama = False

                break

        print(sama)
```

Langkah – langkah :

1. Pertama, kita buat inputan user untuk memasukkan sebuah tuple dengan fungsi `input()` yang kita konversi dari string menjadi tuple menggunakan fungsi `eval()`, dan kita simpan dalam variabel yang bernama `inp_user`
2. Kemudian, kita cek tipe data dari variabel `inp_user` menggunakan fungsi `isinstance(inp_user,tuple)` untuk memastikan bahwa input yang dimasukkan user adalah tuple
3. Jika input bukan tuple, maka program akan mencetak pesan bahwa input user bukan tuple. Namun jika inputan adalah tuple, maka lanjut ke step selanjutnya
4. Kita inisialisasi variabel sama dengan nilai `True` sebagai asumsi awal bahwa semua anggota tuple adalah sama. Selanjutnya, kita cek panjang tuple jika panjangnya sama dengan 0 maka kita akan memberi tahu user bahwa nilainya dianggap `true` (dengan print variabel sama) namun diikuti pemberitahuan bahwa tuple mu kosong. Jika panjang tuple sama dengan 1, maka program akan memberitahu bahwa nilai dalam tuple dianggap sama namun isi tuple nya hanya satu.
5. Jika panjang tuple lebih dari 1 maka program masuk dalam perulangan `for` dengan rentang 0 sampai `len(inp_user)-1` untuk membandingkan elemen-elemen tuple.
6. Dalam perulangan, kita bandingkan elemen ke-1 dengan elemen ke(`i+1`) menggunakan operator `!=` (tidak sama dengan)
7. Jika ditemukan ada dua elemen yang berurutan dan tidak sama maka variabel sama akan kita ubah nilainya menjadi `false` dan perulangan langsung dihentikan (`break`).
8. Artinya jika perulangan selesai tanpa menemukan ketidaksamaan maka variabel sama tetap bernilai `True`
9. Setelah perulangan selesai, program akan mencetak nilai variabel sama yang berisi `True` (jika semua anggota sama) dan `false` (jika ada anggota yang berbeda).

Output :

```

PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu13> & "ta/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/71251189/minggu13/soal01.py
Masukan tuple angka : (90, 90, 90, 90)
True
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu13> & "ta/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/71251189/minggu13/soal01.py
Masukan tuple angka : (40,60,70)
False
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu13> & "ta/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/71251189/minggu13/soal01.py
Masukan tuple angka : ()
True
(Tapi Tuple mu kosong)
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu13> & "ta/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/71251189/minggu13/soal01.py
Masukan tuple angka : (90,)
True
(Tapi Tuple mu hanya memiliki 1 angka)
PS C:\prak_alpro\71251189_alicia\Praktikum-Alpro---71251189\minggu13> & "ta/Local/Programs/Python/Python313/python.exe" c:/prak_alpro/71251189_alicia/71251189/minggu13/soal01.py
Masukan tuple angka : 90
inputmu bukan tuple heii

```

Gambar 2.1 : Output program soal nomor 1

```

soal01.py > ...
1  inp_user = eval(input("Masukan tuple angka : "))
2
3  if not isinstance(inp_user, tuple) :
4      print("inputmu bukan tuple heii")
5  else :
6      sama = True
7      if len(inp_user) == 0 :
8          print(sama)
9          print("(Tapi Tuple mu kosong)")
10     elif len(inp_user) == 1 :
11         print(sama)
12         print("(Tapi Tuple mu hanya memiliki 1 angka)")
13     else :
14         for i in range (len(inp_user)-1) :
15             if inp_user[i] != inp_user[i+1] :
16                 sama = False
17                 break
18         print(sama)
19

```

Gambar 2.2 : Kode Program soal nomor 1

SOAL 2

Source Code :

```

data = ('Alicia Luna Santoso', '71251189', 'Muntilan, Magelang')

nama,nim,alamat = data

tpl_nim = tuple(nim)

nama_depan = nama.split()[0]

```

```

tpl_depan = tuple(nama_depan[1:])

nama_split = nama.split()

tpl_balik = tuple(nama_split[::-1])

print(f>Data : {data}")

print()

print(f"NIM : {nim}")

print(f"NAMA : {nama}")

print(f"ALAMAT : {alamat}")

print()

print(f"NIM : {tpl_nim}")

print()

print(f"NAMA DEPAN : {tpl_depan}")

print()

print(f"NAMA TERBALIK : {tpl_balik}")

```

Langkah – langkah :

1. Pertama, kita buat tuple dengan variabel bernama data yang berisi tiga elemen yaitu nama lengkap, NIM, dan alamat sesuai dengan data kita,
2. Kedua, kita melakukan pemecahan tuple dengan menetapkan variabel nama, nim, dan alamat secara berurutan dari tuple data
3. Ketiga kita membuat variabel bernama tpl_nim yang berfungsi untuk menampung konversi string nim menjadi tuple, sehingga setiap karakter angka pada nim menjadi elemen terpisah dalam tuple
4. Keempat, kita mengambil nama depan dari variabel nama dengan menggunakan split()[0] yang berfungsi untuk memecah string nama menjadi potongan-potongan kata berdasarkan spasi dan kita ambil indeks ke 0 nya. Kita simpan dalam variabel yang bernama nama_depan

5. Selanjutnya, kita buat tuple bernama `tpl_depan` dengan cara mengambil karakter dari indeks ke-1 samapai akhir dari string nama deoan menggunakan slicing `[1:]`, kemudian mengkonversi menjadi tuple menggunakan fungsi `tuple()`
6. Kemudiam, kita pecah string nama menjadi list of kata menggunakan metode `split` dan menyimpannya dalam variabel `nama_split`
7. Ketujuh, buat tuple bernama `tpl_balik` degan cara membalik urutan list `nama_split` menggunakan slicing `[::-1]` kemudian mengkonversi menjadi tuple menggunakan `tuple()`
8. Kemutian kita cetak data, baris kosong, kemudian kita cetak `nim`, `nama`, `alamat` kemudian baris kosong lagi
9. Kita cetak `nim` yang sudah dipisah setiap angka di dalamnya, `nama depan`, dan urutan `nama terbalik`, dimana di setiap bagian diberi jarak satu baris

Output :

```
"C:/Users/alc 357/AppData/Local/Programs/Python/Python313/python.e
c:/prak_alpro/71251189_alicia/Praktikum-Alpro---71251189/minggu13/s
2.py
Data : ('Alicia Luna Santoso', '71251189', 'Muntilan, Magelang')

NIM : 71251189
NAMA : Alicia Luna Santoso
ALAMAT : untilan, Magelang

NIM : ('7', '1', '2', '5', '1', '1', '8', '9')

NAMA DEPAN : ('l', 'i', 'c', 'i', 'a')

NAMA TERBALIK : ('Santoso', 'Luna', 'Alicia')
```

Gambar 2.3 : Output dari soal nomor 2

```
1 data = ('Alicia Luna Santoso', '71251189', 'Muntilan, Magelang')
2 nama, nim, alamat = data
3 tpl_nim = tuple(nim)
4 nama_depan = nama.split()[0]
5 tpl_depan = tuple(nama_depan[1:])
6 nama_split = nama.split()
7 tpl_balik = tuple(nama_split[::-1])
8 print(f'Data : {data}')
9 print()
10 print(f'NIM : {nim}')
11 print(f'NAMA : {nama}')
12 print(f'ALAMAT : {alamat}')
13 print()
14 print(f'NIM : {tpl_nim}')
15 print()
16 print(f'NAMA DEPAN : {tpl_depan}')
17 print()
18 print(f'NAMA TERBALIK : {tpl_balik}')
--
```

Gambar 2.4 : Kode soal nomor 2

SOAL 3

Source Code :

```
nama_file = input("Enter a file name: ")

try :

    handle = open(nama_file)

    hitung_jam = {}

    for baris in handle :

        if baris.startswith('From ') :

            kata = baris.split()

            if len(kata) >= 6 :

                waktu = kata[5]

                jam = waktu.split(':')[0]

                if jam not in hitung_jam :

                    hitung_jam[jam] = 1

                else :

                    hitung_jam[jam] += 1

    handle.close()

    for jam in sorted(hitung_jam.keys()) :

        print(jam, hitung_jam[jam])

except FileNotFoundError :

    print("file tidak ditemukan")
```

Langkah – langkah :

1. Pertama, kita meminta user untuk memasukkan nama file yang akan diproses menggunakan fungsi `input()` dan menyimpan dalam variabel bernama `nama_file`
2. Kemudian, kita buat blok `try-except` untuk menangani kemungkinan error saat membuka file,
3. Ketiga, dalam blok `try`, kita buka file dengan nama sesuai dengan input user menggunakan `open(nama_file)` dan menyimpan dalam variabel `handle`
4. Kemudian, kita buat dictionary kosong dengan nama `hitung_jam` yang akan digunakan untuk menyimpan pasangan jam (sebagai `key`) dan jumlah pesan (sebagai `value`)
5. Kelima, kita lakukan perulangan `for` baris in `handle` untuk membaca setiap baris dalam file satu per satu
6. Keenam, dalam perulangan, program memeriksa apakah baris dimulai dengan string “From “ menggunakan metode `startswith()`. Hanya baris dengan awalan tersebut yang berisi informasi pengirim email
7. Ketujuh, jika ditemukan, program akan memecah baris tersebut menjadi list of words menggunakan metode `split()` dan menyimpan dalam variabel `kata`
8. Kedelapan, program memeriksa apakah panjang list kata minimal 6 elemen untuk memastikan baris memiliki format yang lengkap
9. Kemudian, kita ambil elemen ke 6 (indeks ke 5) dengan memecah string menggunakan `split(":")` kemudian mengambil elemen pertama sebagai jam dan menyimpannya dalam variabel `jam`.
10. Selanjutnya, kita cek apakah variabel `jam` sudah ada sebagai `key` dalam dictionary `hitung_jam`. Jika belum kita tambahkan `key` baru dengan nilai sebanyak 1. Jika sudah ada, kita tambahkan nilainya dengan 1
11. Berikutnya, setelah perulangan `for` selesai kita tutup file menggunakan `handle.close()`

12. Kemudian, program melakukan perulangan untuk mengurutkan dan mengakses setiap key dalam dictionary yang sudah urut dari nilai yang terkecil ke terbesar
13. Dalam perulangan, program mencetak jam dan jumlah pesasn yang masuk pada jam tersebut
14. Jika terjadi error, karena tidak ditemukan, pada bagian except akan memberitahu user bahwa file tidak ditemukan.

Output :

```
Masukan nama file : mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Gambar 2.5 : Output soal no 3

```
soal03.py > ...
1  nama_file = input("Enter a file name: ")
2
3  try :
4      handle = open(nama_file)
5      hitung_jam = {}
6      for baris in handle :
7          if baris.startswith('From ') :
8              kata = baris.split()
9              if len(kata) >= 6 :
10                 waktu = kata[5]
11                 jam = waktu.split(':')[0]
12                 if jam not in hitung_jam :
13                     hitung_jam[jam] = 1
14                 else :
15                     hitung_jam[jam] += 1
16             handle.close()
17
18     for jam in sorted(hitung_jam.keys()) :
19         print(jam, hitung_jam[jam])
20
21 except FileNotFoundError :
22     print("file tidak ditemukan")
```

Gambar 2.6 : Kode Program soal nomor 3